Monodromy Computation Model

Xiaoyang Liu*
University of Waterloo
x98liu@uwaterloo.ca

November 2026

Abstract

A closed path can return to its starting point while analytic continuation reaches another branch. We construct a finite computational calculus in which retained logarithmic branches supply memory, analytic observations select operations, and descent through a covering exposes deeper memory. Three branch coordinates suffice, and every finite observation uses the same cubic power covering. Each program defines a globally real analytic return map whose integer-lattice execution is realized by actual path lifting. Our first theorem simulates every finite-alphabet, finite-control Post–Turing machine, preserving complete configurations, halting, and divergence. Our second theorem identifies the partial numeric functions of the calculus, under fixed primitive-recursive input and output maps, with exactly the partial recursive functions. Deleting descent leaves a Forward language with a syntax-derived finite observation quotient and a primitive-recursive halting predicate. This yields a strict separation within the calculus and identifies access to deeper memory as its decisive additional operation. The constructions and semantic correspondences are formalized in Lean 4 with Mathlib.

1 Introduction

Digital computation is described by discrete state transitions. Continuous-time models such as Shannon's general-purpose analog computer [11] and real-arithmetic models such as the Blum–Shub–Smale machine [1] organize computation through different state spaces and primitive operations. Here the organizing structure is branch transport under analytic continuation.

A closed path can change an analytic branch. Continuing a logarithm around the origin changes its value by $2\pi i$ times the winding number. The retained branch records this integer, and endpoint-fixed homotopies preserve its transport [9, §1.3]. Can this geometry support memory, conditional execution, and universal computation within a finite effective calculus?

We introduce the *Monodromy computation calculus*. Logarithmic branches retain integers; a cubic covering observes their residues; analytic selectors turn those observations into internal control. Affine writing and covering-level descent expose successive digits of memory. Finite expressions determine each return and its stopping signal, and generate paths whose actual lifts reach the next state. The contributions are:

1. A finite calculus with three retained branch coordinates, integer affine updates, analytic selection, descent, and internal feedback. Each program has a globally real analytic return map and a path-lifting realization of every finite run.

*Source code: `https://github.com/ERHUTUZI123/monodromy`.

2. **Theorem I:** a native simulation of every finite-alphabet, finite-control Post–Turing machine, preserving each primitive step, the complete configuration, halting, and divergence.

3. **Theorem II:** under fixed primitive-recursive input and output maps, $\mathcal{C}_{\text{Mono}} = \mathcal{R}_{\text{partial}}$. Deleting descent gives a Forward language with primitive-recursive halting. An undecidable partial-recursive domain proves strict separation between the two languages.

4. A Lean 4 formalization of the definitions, compilation, complete-execution correspondences, and structural separation.

Analytic simulation of Turing machines has a substantial history. Koiran and Moore [10] use integer encodings, analytic selectors, and radix division to obtain universal analytic maps. Graça, Campagnolo, and Buescu [8] develop robust analytic simulations, and Graça and Zhong [7] obtain robust one-dimensional analytic maps. Our contribution organizes these established computational ingredients through a finite covering syntax, actual transport semantics, a fixed effective interface, and a descent deletion theorem. The latter identifies a structural difference between retaining unbounded state and being able to access it in future computation.

Conventions. We write $[a] = \{0, \dots, a-1\}$. Division of integers by a positive integer is Euclidean: $k = B(k \div B) + (k \bmod B)$, with $0 \le k \bmod B < B$, also when $k < 0$. A value of Part A is either a result in A or undefined. For a partial value v, the expression $f_* v$ means its image under a total map f; it is undefined exactly when v is undefined. Thus equalities in Part A preserve both results and divergence.

2 Mathematical Preliminaries

2.1 Transport and logarithmic memory

A covering supplies a precise meaning for carrying a branch. A map $p : E \to X$ is a covering when every point of X has an open neighborhood whose preimage is a disjoint union of open sets, each mapped homeomorphically onto that neighborhood. For a path $\gamma : [0,1] \to X$ and each $e \in p^{-1}(\gamma(0))$, there is a unique lift starting at e. Its endpoint defines transport

$$\mathcal{T}_\gamma : p^{-1}(\gamma(0)) \longrightarrow p^{-1}(\gamma(1)).$$

If $\gamma * \delta$ means first γ, then δ, the standard lifting theorems give [9, §1.3]

$$\mathcal{T}_{\gamma*\delta} = \mathcal{T}_\delta \circ \mathcal{T}_\gamma, \qquad \mathcal{T}_{\bar{\gamma}} = \mathcal{T}_\gamma^{-1}. \tag{2.1}$$

Endpoint-fixed homotopies preserve transport. For loops, these endpoint maps form the monodromy action on a fiber.

For the complex exponential $\exp : \mathbb{C} \to \mathbb{C}^*$ and the power maps on $\mathbb{C}^*$, local inverse branches are holomorphic. Carrying such a branch along a path is analytic continuation, represented by the path's lift. A Riemann surface separates different branches into distinct points over a common base [6]. The construction below uses the explicit exponential and power coverings, together with their regular-root selectors.

For memory we use the exponential's real angular restriction

$$p : \mathbb{R} \longrightarrow \mathbb{S}^1, \qquad p(\theta) = e^{i\theta}. \tag{2.2}$$

Its fiber above 1 is $2\pi\mathbb{Z}$. These are the angular representatives of logarithmic branches. We use the normalized coordinate $k \in \mathbb{Z}$ for the retained angle $2\pi k$. The loop $t \mapsto e^{2\pi i n t}$ transports k to $k+n$: its lift is $t \mapsto 2\pi(k+nt)$. In particular, a fixed loop acts by the same translation on every branch.

Retaining a point of the fiber is essential. The common base value 1 alone contains no record of the integer k. Several retained branches will be represented by a vector $x \in \mathbb{Z}^n$.

2.2 A finite observation of an infinite branch

To inspect one digit of a retained integer, factor the exponential covering through a power covering. For $B \geq 1$, put

$$r_B(x) = e^{2\pi i x/B}, \qquad P_B(z) = z^B \quad (z \in \mathbb{C}^*). \qquad P_B(r_B(x)) = e^{2\pi i x}. \tag{2.3}$$

For integer x, the observed root $r_B(x)$ depends exactly on $x \bmod B$. The state retains all of x, while this observation has only B values. Throughout the computational calculus we fix $B = 3$.

This observation is itself compatible with path lifting. For a continuous real curve $u(t)$, $r_B(u(t))$ is the lift through P_B of $e^{2\pi i u(t)}$ starting at $r_B(u(0))$. Uniqueness of lifting proves that its endpoint is $r_B(u(1))$. Each expression using an observation will have its own induced curve of this form.

2.3 Regular roots as analytic selectors

A finite observation becomes a choice of operation through an analytic interpolant. Suppose $f : \mathbb{C} \to \mathbb{C}$ is entire and r is a regular point, so $f'(r) \neq 0$. Define the normalized divided-difference kernel

$$K_r^f(z) = \frac{f(z) - f(r)}{f'(r)(z-r)}, \qquad K_r^f(r) = 1. \tag{2.4}$$

The singularity at r is removable. Consequently K_r^f is entire; on the fiber $f^{-1}(f(r))$ it equals 1 at r and 0 at every other point. If $r_0, \ldots, r_{B-1}$ are distinct regular roots in one fiber, then $\sum_d v_d K_{r_d}^f(z)$ selects v_j at $z = r_j$.

For the cubic cover, let $\zeta_d = e^{2\pi i d/3}$, $d \in [3]$. The kernels and their real observations are

$$K_d(z) = \frac{z^2 + \zeta_d z + \zeta_d^2}{3\zeta_d^2}, \qquad W_d(x) = \operatorname{Re} K_d(r_3(x)). \tag{2.5}$$

The polynomial formula includes the removable value in (2.4). Each W_d is real analytic on $\mathbb{R}$, and

$$W_d(k) = \begin{cases} 1 & k \bmod 3 = d, \\ 0 & k \bmod 3 \neq d \end{cases} \qquad (k \in \mathbb{Z}). \tag{2.6}$$

Thus a finite choice is an analytic formula, evaluated at the current retained state.

The kernels also respect hierarchical composition. For entire f, g with $f'(r) \neq 0$ and $g'(f(r)) \neq 0$, the divided-difference chain rule gives

$$K_r^{g \circ f}(z) = K_{f(r)}^g(f(z)) K_r^f(z). \tag{2.7}$$

When $f(z) \neq f(r)$ the factors telescope. When $z \neq r$ and $f(z) = f(r)$ both sides vanish, and at $z = r$ both sides equal one. This is a normalized instance of the classical chain rule for divided differences [5]. It explains how selection at one level can be refined by selection at the next. Our finite syntax will implement the hierarchy by nested cubic queries.

3 Syntax and Semantics of the Monodromy Calculus

The universal calculus retains three normalized logarithmic coordinates $x = (x_0, x_1, x_2) \in \mathbb{Z}^3$, representing the angles $2\pi x_j$ above the common base point 1. In the Turing realization, x_0 stores finite control and the current symbol, and x_1, x_2 store two words. We first describe the memory operations, then give their finite syntax for an arbitrary finite number n of coordinates. Both main theorems use $n = 3$.

3.1 Memory, writing, and descent

The selectors provide both a branch query and its companion descent:

$$S(x; y_0, y_1, y_2) = \sum_{d=0}^{2} W_d(x) y_d, \qquad D(x) = \frac{x - \sum_{d=0}^{2} d W_d(x)}{3}. \tag{3.1}$$

For integers, these formulas satisfy

$$S(k; y_0, y_1, y_2) = y_{k \bmod 3}, \qquad D(k) = k \div 3. \tag{3.2}$$

The second equality is the Euclidean decomposition of k. Descent removes a digit whose value has first been observed; applying the same observation to the result reveals the following digit.

For binary words, reserve zero for the empty word and use the nonzero ternary digits for letters:

$$E([]) = 0, \qquad E(b :: w) = b + 1 + 3E(w), \qquad b \in \{0, 1\}. \tag{3.3}$$

This is injective. Indeed, the residue modulo three distinguishes an empty word from its first letter, and division by three recovers the tail; induction recovers the whole word. Prepending b is the affine write $x \mapsto b + 1 + 3x$, and D removes the first letter. For example,

$$E([0, 1]) = 7, \qquad 7 \xrightarrow{\text{read } 1} D(7) = 2 \xrightarrow{\text{read } 2} D(2) = 0.$$

The two observations identify the letters 0 and 1, followed by the empty word. The same cubic covering is used throughout.

3.2 Finite expressions and guards

A program must give a finite description of every operation it can perform. Fix $n \geq 1$ coordinates. Its expressions are generated by

$$t ::= x_j \mid c \mid t + t \mid c\,t \mid \mathrm{sel}(t; t_0, t_1, t_2) \mid D(t), \qquad j \in [n], \quad c \in \mathbb{Z}. \tag{3.4}$$

The integer interpretation $\mathrm{ev}_{\mathbb{Z}}(t, x)$ uses ordinary integer arithmetic, selects branch $\mathrm{ev}_{\mathbb{Z}}(t, x) \bmod 3$, and interprets D as Euclidean division by three. The real interpretation $\mathrm{ev}_{\mathbb{R}}(t, u)$ uses the same arithmetic and the formulas

$$\mathrm{ev}_{\mathbb{R}}(\mathrm{sel}(t; t_0, t_1, t_2), u) = \sum_{d=0}^{2} W_d(\mathrm{ev}_{\mathbb{R}}(t, u))\, \mathrm{ev}_{\mathbb{R}}(t_d, u), \tag{3.5}$$

$$\mathrm{ev}_{\mathbb{R}}(D(t), u) = D(\mathrm{ev}_{\mathbb{R}}(t, u)). \tag{3.6}$$

Both the key and the possible outcomes can depend on the whole current state. All expressions are finite trees.

Stopping is specified in the same language of finite observations. A guard is generated by

$$h ::= \mathsf{false} \mid \mathsf{true} \mid \mathrm{sel}(t; h_0, h_1, h_2).$$

Its integer interpretation is Boolean. Replacing its constant leaves by 0 and 1 gives an expression $\widehat{h}$ and hence a real analytic indicator $H(u) = \mathrm{ev}_{\mathbb{R}}(\widehat{h}, u)$. For a guard h and expressions a, b, define $\mathrm{gate}(h; a, b)$ by replacing each true leaf of h by a and each false leaf by b, preserving all query nodes. This is again an expression of (3.4).

Lemma 3.1 (Exact analytic interpretation)**.** *Every expression defines a real analytic function on $\mathbb{R}^n$, and*

$$\mathrm{ev}_{\mathbb{R}}(t, x) = \mathrm{ev}_{\mathbb{Z}}(t, x) \qquad (x \in \mathbb{Z}^n),$$

where integers on the right are embedded in $\mathbb{R}$. A guard indicator is 1 exactly at its true integer states and 0 at its false integer states. Moreover,

$$\mathrm{ev}_{\mathbb{Z}}(\mathrm{gate}(h; a, b), x) = \begin{cases} \mathrm{ev}_{\mathbb{Z}}(a, x) & h(x) = \mathsf{true}, \\ \mathrm{ev}_{\mathbb{Z}}(b, x) & h(x) = \mathsf{false}. \end{cases}$$

Proof. Use simultaneous structural induction. Coordinates and constants are analytic, and sums and products preserve analyticity. A query is the finite sum (3.5) of analytic compositions and products; descent is (3.6). On integer states, each induction hypothesis makes the key an integer, so (3.2) gives the required query or quotient. The guard and gate assertions follow by induction on the guard tree. □

3.3 Returns, stopping, and complete execution

The program consists of a guard and an update expression for each coordinate.

Definition 3.2 (Program and execution)**.** A program $P = (h, u_0, \ldots, u_{n-1})$ has returning expressions $r_j = \mathrm{gate}(h; x_j, u_j)$. Write

$$T_P(x)_j = \mathrm{ev}_{\mathbb{Z}}(r_j, x), \qquad F_P(v)_j = \mathrm{ev}_{\mathbb{R}}(r_j, v). \tag{3.7}$$

Its partial step stops at x when $h(x)$ is true, and otherwise continues at $T_P(x)$. The complete result $\mathcal{E}_P(x) \in \mathrm{Part}\,\mathbb{Z}^n$ is $T_P^k(x)$ at the first $k \geq 0$ with $h(T_P^k(x))$ true, and is undefined if no such k exists.

By Lemma 3.1, F_P is globally jointly real analytic, preserves the integer lattice, and restricts there to T_P. At a stopped integer state $T_P(x) = x$, so returns can be counted after halting by keeping the state stationary. The native memory domain is the exact lattice $\mathbb{Z}^n$ inside $\mathbb{R}^n$, with its real coordinates and retained exponential branches. Its native step tests the internally generated indicator $H = 1$ and otherwise applies F_P.

Let ι be the inclusion of an integer state into this native domain. Induction on each step and the first stopping time yields the full partial-value equality

$$\mathcal{E}_P^{\mathrm{native}}(\iota x) = \iota_* \mathcal{E}_P(x). \tag{3.8}$$

The same induction shows that an infinite integer run has an infinite native run. Conversely, every native step on this domain is the image of the integer step. Thus (3.8) also preserves divergence.

The guard, branch choices, and writes are functions of the current state and the fixed finite syntax. That syntax is the runtime controller; a source Turing table enters when the program is constructed. Execution consists of discrete returns, with the stopping signal read at those returns. The following path generator realizes each return.

3.4 Actual continuation paths

We now connect a return to transport rather than merely identifying its endpoint symbolically. For a native state $x \in \mathbb{Z}^n$, define the coordinate angle curves and their base loops by

$$\ell_{P,x,j}(t) = 2\pi\big((1-t)x_j + tF_P(x)_j\big), \qquad \gamma_{P,x,j}(t) = e^{i\ell_{P,x,j}(t)}. \tag{3.9}$$

Both endpoints of $\gamma_{P,x,j}$ are 1. The retained starting angle is $2\pi x_j$. The curve $\ell_{P,x,j}$ is a lift of the generated loop with this starting point, so uniqueness of lifting gives

$$\mathcal{T}_{\gamma_{P,x,j}}(2\pi x_j) = 2\pi T_P(x)_j. \tag{3.10}$$

The program's internal analytic rules first determine its return; the interpolation in (3.9) then generates a path realizing that return through the exponential covering.

Proposition 3.3 (Finite-run lifting)**.** *Concatenate the generated loops for k consecutive returns, updating the state after each return. The resulting path $\Gamma_{P,k,x,j}$ satisfies*

$$\mathcal{T}_{\Gamma_{P,k,x,j}}(2\pi x_j) = 2\pi(T_P^k(x))_j. \tag{3.11}$$

Every query node also follows actual transport through the cubic power covering along its own induced base path.

Proof. Equation (3.10) proves the case of one return, and transport composition (2.1) proves the finite-run statement by induction. For a query key t along a continuous state curve $v(s)$, use the continuous real curve $u(s) = \mathrm{ev}_{\mathbb{R}}(t, v(s))$. Equation (2.3) identifies $r_3(u(s))$ as the unique lifted query curve. Its endpoint at an integer state is the root indexed by $\mathrm{ev}_{\mathbb{Z}}(t, x) \bmod 3$. □

Endpoint-fixed homotopies of the generated base paths preserve these transport results. This statement concerns the path realization of a specified return. Its computational meaning comes from the finite analytic rule that generated the path.

4 Theorem I: Turing Universality

4.1 Arbitrary finite tables from cubic queries

Repeated observation and descent read an arbitrary finite ternary address. Let $Q_d(t; A)$ be the expression querying a table A with 3^d entries. Define

$$\begin{aligned} Q_0(t; A) &= A_0, \\ Q_{d+1}(t; A) &= \mathrm{sel}\big(t; Q_d(D(t); (A_{0+3j})_j), Q_d(D(t); (A_{1+3j})_j), Q_d(D(t); (A_{2+3j})_j)\big). \end{aligned} \tag{4.1}$$

Induction using $k = (k \bmod 3) + 3(k \div 3)$ proves

$$\mathrm{ev}_{\mathbb{Z}}(Q_d(t; A), x) = \mathrm{ev}_{\mathbb{Z}}(A_{\mathrm{ev}_{\mathbb{Z}}(t,x) \bmod 3^d}, x). \tag{4.2}$$

The same construction works for guards. Every tree is finite, and every node uses the same cubic query or descent.

For an alphabet $[a]$ with $a \geq 1$, encode letter j as the binary word of length a whose only 1 is at position j. Its ternary block value is

$$\delta_j = \sum_{i=0}^{a-1} (1 + \mathbf{1}_{i=j}) 3^i, \quad B = 3^a, \qquad E_a([]) = 0, \quad E_a(j :: w) = \delta_j + BE_a(w). \tag{4.3}$$

The δ_j are distinct and satisfy $0 < \delta_j < B$. A query of depth a therefore reads the first letter, including the empty-word sentinel, and a descents expose the tail. The recurrence proves injectivity of E_a by the same residue-and-quotient argument as for (3.3). This block encoding accommodates any finite alphabet while keeping the covering degree fixed.

4.2 Two stacks and a finite controller

We use the Post–Turing convention formalized as Mathlib's TM_0 [3]. A table has a finite alphabet Σ with a distinguished blank and a finite control set Q with an initial label. Each pair (q, s) either halts or specifies a new label together with one primitive instruction: write a symbol, move left, or move right. A conventional write-and-move instruction is represented by a finite sequence of these primitives.

A complete configuration can be represented by

$$c = (q, L, s, R),$$

where L and R are finite words, nearest symbols first, and s is the current symbol. Words represent blank-extended half-tapes: appending trailing blanks gives the same half-tape. Every finitely supported tape has such a representative. Let head_0 return the first symbol, or the blank on an empty word. A right move with target label q' is

$$(q, L, s, R) \longmapsto (q', s :: L, \mathrm{head}_0(R), \mathrm{tail}(R)). \tag{4.4}$$

A left move is symmetric, and writing changes only s and the label. These operations induce exactly the usual operations on the entire blank-extended tape.

Choose blank-preserving enumerations $\Sigma \simeq [a]$ and $Q \simeq [q]$, with the blank and initial label numbered zero. Encode a stack configuration by three integers:

$$C(q_0, L, s, R) = \big(aq_0 + s,\ E_a(L),\ E_a(R)\big). \tag{4.5}$$

The first coordinate jointly stores the finite controller and the current symbol; the other coordinates store the two unbounded words.

The integer return row for a right move is explicit. If $x = (c, l, r)$ encodes (q_0, L, s, R), set

$$(c, l, r) \longmapsto \big(aq' + \mathrm{head}_0(r),\ \delta_s + Bl,\ r \div B\big), \tag{4.6}$$

where $\mathrm{head}_0(r)$ is a finite table on $r \bmod B$, returning the decoded letter at a valid block and blank at zero. Its other entries can be assigned arbitrarily, for instance blank. Equation (4.2) implements this table. A left row exchanges the stack roles, and a write row is $(aq' + s', l, r)$. Thus every row belongs to the finite cubic syntax.

There are aq possible control-symbol pairs. A table query of depth aq selects the corresponding row from coordinate c; indeed $aq < 3^{aq}$. Addresses outside the source table are stationary halting entries. A guard tree with the same address reports exactly the halting entries of the source. The compiled program is denoted P_M.

Lemma 4.1 (One-step compilation)**.** *For every stack configuration c, including halting ones,*

$$\mathrm{step}_{P_M}(C(c)) = C_* \,\mathrm{step}_M(c). \tag{4.7}$$

Here both sides are optional successors: a halting step has no successor. The code C is injective on stack configurations.

Proof. The control query selects the entry indexed by $aq_0 + s$. For a write, the first coordinate changes to the prescribed label and symbol and both stacks stay fixed. For a right move, $E_a(s :: L) = \delta_s + BE_a(L)$, while $E_a(R) \div B = E_a(\operatorname{tail} R)$ and its block residue recovers $\operatorname{head}_0(R)$. These are exactly (4.4) and (4.6). The left case is symmetric. The compiled guard is true exactly when the source entry halts. Finally, the first code coordinate recovers (q_0, s) by finite product enumeration and the other two recover their words by injectivity of E_a. □

4.3 The full simulation theorem

Theorem 4.2 (Theorem I: Turing simulation)**.** *For every finite-alphabet, finite-control Post–Turing machine M, there is a program P_M with three retained logarithmic branches and only cubic queries and descents, with the following properties. Every complete source configuration has a representing native state. From every such representation, one source primitive step is one native return. For each number of returns, the native state represents the entire source configuration, with halting configurations kept stationary. Complete execution preserves the final configuration, halting, and divergence in both directions. Every finite native run is realized by the actual transports in* (3.11)*, and its return map is globally jointly real analytic.*

Proof. First enumerate the finite alphabet and control, preserving their distinguished elements. A complete source tape has representatives L, R as above. Initialize the native state with $C(q_0, L, s, R)$. Lemma 4.1 and induction give the exact stack execution at every return. Passing to blank-extended half-tapes commutes with each move and write, so this execution represents the whole source tape, independently of trailing-blank representatives.

At any represented configuration, the source has no successor exactly when the native guard is true. Consequently, a source halt yields a native halt at the same return with the represented final configuration. Conversely, the first native halt is a source halt, since its guard is precisely the source table's stopping predicate. If either run continues forever, this equivalence at every finite return forces the other to continue forever. Equivalently, on stack representatives the complete partial values satisfy

$$\mathcal{E}_{P_M}(C(c)) = C_* \mathcal{E}_M^{\mathrm{stack}}(c), \tag{4.8}$$

and decoding the right side gives the complete source result. Equation (3.8) transfers this equality to native execution. Lemma 3.1 gives global analyticity, and Proposition 3.3 supplies each finite run's actual lifted paths. □

4.4 A complete computation

Consider a two-symbol alphabet [2] with blank 0, two controls $q_0 = 0, q_1 = 1$, and the table

$$(q_0, 0) \mapsto (q_1, \text{write } 1), \qquad (q_1, 1) \mapsto (q_0, \text{move right}),$$

with all other entries halting. Start with current symbol 0 and right word [1]. For $a = 2$, the blocks of letters 0 and 1 are $[1, 0]$ and $[0, 1]$, so $\delta_0 = 5$, $\delta_1 = 7$, and $B = 9$. The entire run is

Return	Source configuration	Integer state	Action
0	$(q_0, [], 0, [1])$	$(0, 0, 7)$	write 1
1	$(q_1, [], 1, [1])$	$(3, 0, 7)$	move right
2	$(q_0, [1], 1, [])$	$(1, 7, 0)$	halt

The first return changes the control coordinate by 3. The second changes the three coordinates by $(-2, 7, -7)$. Thus their exponential base loops have these respective integer windings; lifting from the retained branches gives exactly the displayed states. The halting guard is generated by the same finite control query as the updates. This example is a direct instance of the compilation rules; the general theorem proves the same correspondence for arbitrary finite tables and unbounded word lengths.

5 Theorem II: Characterization and Sublanguage Separation

5.1 A fixed numeric interface

A computational characterization must account for the encoding of both inputs and results. We fix once and for all two total maps

$$I : \mathbb{N} \longrightarrow \mathbb{Z}^3, \qquad O : \mathbb{Z}^3 \longrightarrow \mathbb{N}. \tag{5.1}$$

They are independent of the requested function and the program. The input map prepares a canonical binary numeric tape over the fixed alphabet used by Mathlib's partial-recursive-to-Turing translation. It then stores the current symbol, an empty left word, and the encoded right word in the three coordinates. The output map decodes only the right word, reads its main-track bits in order, and ignores separators and blank symbols. Appendix A specifies both maps, including their values on malformed integer states.

Lemma 5.1 (Effective interface)**.** *The fixed maps I and O are primitive recursive. If a completed configuration is the canonical numeric result for n, then its native code has output n.*

Proof. Finite-symbol encoding, binary numeral construction, list operations, and the recurrence (4.3) are primitive recursive. For decoding a natural code v, scan its lowest block and replace v by $v \div 3^a$. A valid nonempty block code decreases, and the encoded number bounds the word length. Running exactly the original v scan steps, with zero or an invalid block ending the scan, gives a total primitive-recursive decoder. On valid codes it inverts E_a. Clamp negative integer coordinates to zero. The output fold replaces an accumulator z by $2z + b$ at a bit b and leaves it unchanged at every other symbol. It is primitive recursive and reads the canonical numeric tape as n. Trailing blanks leave the fold unchanged, so the result also holds for any representative of the same half-tape. □

Definition 5.2 (Realized numeric function)**.** A program P computes $f : \mathbb{N} \to \mathrm{Part}\,\mathbb{N}$ when, for every n,

$$O_* \mathcal{E}_P(I(n)) = f(n). \tag{5.2}$$

Let $\mathcal{C}_{\mathrm{Mono}}$ consist of the functions so computed by three-coordinate programs of (3.4). Equivalently, by (3.8), the native execution has a result representing an output m exactly when m is the result of $f(n)$.

5.2 Upper bound: effective syntax

Lemma 5.3 (Effectiveness of finite execution)**.** *For each fixed finite program P, its integer next-state map and guard are primitive recursive. Its complete execution $\mathcal{E}_P$ is partial recursive under a standard finite-vector integer encoding.*

Proof. Integer addition, multiplication by a fixed integer, and Euclidean division and remainder by a positive fixed integer are primitive recursive in the signed integer encoding. Structural induction on expressions therefore proves the claim for their interpretations: a query is a case distinction over three primitive-recursive branch values, and descent is division by three. Induction on guards proves their claim, including the gate used for the returning expressions. The partial step, with its optional successor, is consequently primitive recursive.

Iterate this effective step until it stops. Formally, the loop body returns either the current state as a final result or a successor to iterate. Unbounded iteration of such an effective body is partial recursive. Alternatively, search for the least k with the primitive-recursive predicate $h(T_P^k(x))$ true, and return $T_P^k(x)$. This computes exactly Definition 3.2, with the search undefined precisely on the infinite runs. □

Proposition 5.4 (Computational upper bound)**.** *Every function in $\mathcal{C}_{\text{Mono}}$ is partial recursive.*

Proof. Equation (5.2) expresses the function as the composition of the primitive-recursive input, the partial-recursive execution, and the primitive-recursive output. These classes are closed under the indicated composition. □

5.3 Lower bound: preserving numeric results

For completeness, we describe the classical computation-theoretic bridge used in addition to Theorem 4.2. The partial recursive functions are generated from zero, successor, and projections by composition, primitive recursion, and unbounded search. Each has a finite code and can be executed by a finite-control stack machine on a fixed finite alphabet. Composition stores intermediate results; primitive recursion keeps a counter and accumulator; unbounded search increases a candidate until its defining test returns zero, continuing forever when that computation supplies no result. These constructions are the standard route from partial recursion to Turing computation. Mathlib supplies their formal implementation and the translations from stacks to a tape and then to Post–Turing primitives [2, 3].

The following form records the exact interface needed here. For every partial recursive f, there is a source code c and a finite-support Post–Turing table M_c over one fixed alphabet such that

$$\mathcal{E}_{M_c}(\text{init}(W_n)) = (\text{halt}_c \circ [\,\cdot\,])_* f(n). \tag{5.3}$$

Here W_n is the fixed numeric input word, and $\text{halt}_c([m])$ is the complete canonical final configuration carrying numeric result m. It includes the final label and the entire tape. The translated code visits only finitely many labels: induction over its finite syntax supplies a closed finite support containing the initial label. Restricting to that support makes the control set finite while preserving every reachable step.

The complete partial equality (5.3) follows through the translations, rather than just a stopping equivalence. The stack-to-tape representation determines the whole packed tape from all its track projections. Hence each terminal stack configuration has its specified canonical terminal tape; the later primitive translation preserves that complete configuration. This also gives the converse implication for termination and preserves divergence.

Proposition 5.5 (Computational lower bound)**.** *Every partial recursive $f : \mathbb{N} \to \text{Part}\,\mathbb{N}$ belongs to $\mathcal{C}_{\text{Mono}}$.*

Proof. Choose M_c as in (5.3), restrict to its closed finite support, enumerate its control with the initial label zero, and apply the cubic compilation of Section 4. The source alphabet and W_n are fixed independently of c. Since the initial label is zero, the code of the initial stack configuration is exactly $I(n)$: its first coordinate is just the numbered current symbol,

regardless of the number of control labels. The remaining coordinates are zero and the encoded right word.

The exact partial equalities of the source translations and (4.8) identify every native terminal state with the corresponding canonical result tape. Lemma 5.1 reads that state as m. Thus if $f(n)$ returns m, native execution returns a state read as m; if native execution returns a state, the source has returned, and its unique numeric result is its readout. If $f(n)$ is undefined, the complete-execution equalities force native execution to be undefined as well. This is precisely (5.2). □

Theorem 5.6 (Theorem II: exact characterization)**.** *Under the fixed primitive-recursive maps I and O,*

$$\mathcal{C}_{\mathrm{Mono}} = \mathcal{R}_{\mathrm{partial}}, \tag{5.4}$$

where $\mathcal{R}_{\mathrm{partial}}$ is the class of partial recursive functions $\mathbb{N} \to \mathrm{Part}\,\mathbb{N}$. Each realizing program uses three retained branches and the same degree-three power covering for every finite query and descent. Its native return map is globally jointly real analytic and its finite runs satisfy (3.11)*.*

Proof. Propositions 5.4 and 5.5 give the two inclusions. Lemma 3.1 and Proposition 3.3 supply the analytic and transport properties for the very same realizing program. □

5.4 Forward execution and its finite observation

The characterization establishes the capacity of the whole calculus. To isolate the contribution of descent, call a term, guard, or program *Forward* when every expression in it is built without D. The remaining operations still allow unbounded integer growth and arbitrarily nested finite queries.

Lemma 5.7 (Closed residue observation)**.** *For each Forward program P in n coordinates, one can compute from its syntax a positive integer M such that coordinatewise congruence modulo M is preserved by T_P and determines the guard h.*

Proof. More generally allow any finite query degree $B \geq 1$ in the syntax; the cubic language is its special case. Define a syntactic support certificate at M recursively. Coordinates and constants are supported; sums and integer scaling preserve support of their subterms. A query of degree B is supported when $B \mid M$ and its key and all branches are supported at M. Induction on a certificate proves that the term's value modulo M depends only on the input modulo M: at a query, congruent keys have equal residues modulo B, select the same branch, and give congruent results.

A positive sufficient modulus is obtained recursively: use 1 for coordinates and constants, multiply moduli at sums, preserve the modulus at integer scaling, and at a query multiply B, the key's modulus, and all branch moduli. Induction on support certificates shows that they persist on replacing a modulus by a multiple. The analogous recursion on guard trees makes their Boolean values equal on congruent inputs. Take a common product for the guard and all updates. The guarded return then preserves congruence, since congruent states make the same stopping choice. All these are finite computations on syntax. □

Consequently the observation $\pi_M(x) = (x_j \bmod M)_j$ evolves autonomously on the finite set $(\mathbb{Z}/M\mathbb{Z})^n$. It carries a well-defined successor $\overline{T}_P$ and stopping predicate $\overline{h}$:

$$\pi_M T_P = \overline{T}_P \pi_M, \qquad h = \overline{h}\pi_M.$$

This quotient preserves the whole observed partial execution, including its domain of definition.

Theorem 5.8 (Forward halting and strict separation)**.** *For a Forward program P and a syntax-derived modulus $M > 0$,*

$$\mathcal{E}_P(x) \text{ is defined} \iff \exists k < M^n \; h(T_P^k(x)) = \mathsf{true}. \tag{5.5}$$

Its halting predicate on integer states is primitive recursive. There is a function in $\mathcal{C}_{\text{Mono}}$ computed by no three-coordinate Forward program under the fixed interface I, O.

Proof. The quotient has M^n states. If its first M^n observations contain no stopping state, either they already contain a repetition or the next observation repeats one of them. Determinism then makes all subsequent observations cycle without stopping. This proves (5.5). The finite iteration and guard are primitive recursive by Lemma 5.3, and the bound is a constant for the fixed program, so bounded search proves primitive-recursive halting.

For strictness, enumerate the ordinary partial-recursive codes and let $g(e)$ be the result of the code numbered e on input zero. This is partial recursive and has an undecidable domain by the halting theorem. Theorem 5.6 realizes g in the full calculus. A Forward realization would decide its domain by applying the halting test to the fixed primitive-recursive input $I(e)$, a contradiction. □

The argument uses only a finite autonomous observation carrying the stopping decision. In particular it gives a general principle beyond this syntax.

Corollary 5.9 (Closed observation barrier)**.** *Suppose $T : S \to S$ has an observation $\pi : S \to F$ into a finite set with a fixed finite encoding, $\pi T = \overline{T}\pi$, and stopping is $\overline{h}\pi$. If an input initialization J has a computable observation πJ, then the halting domain on these inputs is decidable.*

Proof. Start at $\pi J(n)$ and inspect the first $|F|$ finite observations. The preceding repetition argument decides whether a stopping observation occurs. The hidden state space S may be infinite. □

5.5 Descent and the depth of observation

Descent breaks the closed residue argument in an exact way. The following result concerns all integers, making its quantifiers independent of any particular word encoding.

Proposition 5.10 (Exact resolution of descent)**.** *Let $B > 0$, $m \geq 2$, $N \in \mathbb{Z}$, and $r \in \mathbb{N}$. Then*

$$[\, \forall x, y \in \mathbb{Z}, \quad x \equiv y \pmod{N} \Longrightarrow x \div B^r \equiv y \div B^r \pmod{m} \,] \iff B^r m \mid N. \tag{5.6}$$

If $B \geq 2$, preservation at every depth r is equivalent to $N = 0$, where congruence modulo zero is equality.

Proof. First prove the one-step assertion for a positive divisor A. If $Am \mid N$, replacing x by $x + Nk$ changes its quotient by Nk/A, a multiple of m. For necessity, write $N = Aq + s$ with $0 \leq s < A$. Comparing the congruent inputs $0, N$ gives $q \equiv 0 \pmod{m}$. If $s > 0$, compare the congruent inputs $A - s$ and $A - s + N$. Their quotients are 0 and $q + 1$, which would give $q + 1 \equiv 0 \pmod{m}$, a contradiction. Thus $s = 0$ and $Am \mid N$. Apply this with $A = B^r$; repeated Euclidean division by B equals division by B^r. If every $B^r m$ divides a nonzero N, then $B^r \leq |N|$ for all r, contradicting unbounded growth of the powers. Conversely, congruent inputs modulo zero are equal and remain equal at every depth. □

For the cubic tower, every further digit removal multiplies the required input modulus by three. Forward execution keeps all future stopping observations inside one finite quotient. Descent permits future queries to depend on successively deeper memory, and the full language uses this access to implement the two stacks. The strict separation is a property of these languages; particular programs containing descent can still halt immediately, for example when their guard is constantly true.

6 Formalization in Lean 4

The development uses Lean 4 [4] and Mathlib [12]. Its proof architecture connects three layers: actual covering-space transport, exact analytic execution on an invariant lattice, and effective discrete computation. The final statements combine these layers for the same finite program.

6.1 Geometric states and dependent fibers

A geometric state has type $\{e : E \mid p(e) = x\}$, so its type records the base point x. A path from x to y transports between these two fiber types. In `Covering.lean`, transport is defined by Mathlib's `IsCoveringMap.monodromy`, applied to the path's endpoint-fixed homotopy class under an explicit covering-map hypothesis. Lift existence and uniqueness identify this operation with an actual continuous lifted curve.

The dependent fiber types make endpoint compatibility a mathematical obligation. A supplied lift carries a continuity proof, its projection equation, and its specified initial value. To identify the terminal fiber element with the intended state, `Lattice.segment_transport` proves equality of their underlying points and uses subtype extensionality. Concatenation then yields the finite-run transport law. Thus the geometric certificate concerns actual lifts with compatible endpoints throughout the run.

6.2 Analytic execution and the grid invariant

The inductive expression type in `CoverExpressions.lean` has integer and real interpretations. Structural induction proves `evalReal_int` and joint real analyticity. The quotient and remainder identities use Euclidean integer division, including negative inputs; the factor 2π connects normalized integers to the actual retained angular branches.

Native memory is a real vector together with a proof of membership in the embedded integer grid. Grid preservation makes the analytic return a map on this subtype. Its value is computed by the real expression interpretation; the integer witness certifies legality. The guard's analytic indicator is proved to agree with the Boolean decision on this same grid. These correspondences give the exact optional step and then `Program.evalReal_init`, the complete partial-execution equality (3.8).

6.3 Complete results and effective representations

The Turing lower bound composes Mathlib's verified stack and tape translations with the cubic compiler. Two additional bridges recover the required complete result. First, equality of all track projections determines the entire packed tape. Second, a fixed source code has a finite transition-closed control support. Restricting labels to this support produces a finite control subtype; each target label carries the membership proof supplied by transition closure. Forgetting these certificates preserves the complete configuration and execution. The pointed enumeration keeps the initial label zero, which gives the program-independent input map.

For the upper bound, primitive-recursive certificates cover signed integer arithmetic, finite expressions, guards, and transitions. Stopped iteration is then partial recursive. The state encoding explicitly uses the encoding associated with its chosen `Primcodable` finite-vector representation. The fixed input and total output maps have separate primitive-recursive proofs. These choices align the numerical interface with the execution semantics in `Structure.continuation_characterization`.

6.4 Source and verification

The Lean development is in `src/Monodromy/`, with root import `src/Monodromy.lean`. Toolchain and dependency versions are recorded in `lean-toolchain`, `lakefile.lean`, and `lake-manifest.json`. Appendix B gives the pinned versions and the correspondence between the paper's statements and formal declarations. From the repository root, verification uses

```
lake build
lake env lean -DautoImplicit=false -DwarningAsError=true src/Audit.lean
```

The audit checks every exported `Monodromy` declaration and its transitive proof dependencies against the standard Lean axioms `propext`, `Quot.sound`, and `Classical.choice`.

7 Discussion and Conclusion

7.1 The computational role of deeper observation

The construction organizes computation around three related structures. An infinite covering fiber retains an unbounded discrete state. A finite covering observation turns selected information into analytic control. Descent changes which part of that retained state becomes available to the same finite observation. Affine writing and finite feedback complete the execution mechanism. The resulting three-coordinate calculus has exactly classical partial-recursive power, with a geometric realization of its finite runs.

The distinction between retention and access matters. An infinite state space by itself can still support only decidable stopping when a closed finite observation governs every future decision. Corollary 5.9 states this obstruction without assuming a particular geometry. In the present calculus, Proposition 5.10 measures exactly how descent escapes a fixed residue observation, and Theorem 5.8 converts that distinction into a separation of computational power.

The covering groups needed here are elementary: the logarithmic memory has integer translations, and the finite power observations have cyclic monodromy. Conditional execution is supplied by state-dependent analytic feedback. Thus this realization already arises from abelian covering actions; its computational organization lies in how retained branches, finite queries, and descent interact. Transport composition and homotopy invariance govern the paths that implement that organization.

7.2 Physical realization and path deformation

Homotopy invariance gives exact protection of branch transport against endpoint-fixed path deformations within the covering base. A physical realization would require a quantitative account of the perturbations that preserve this class, together with reliable branch observation, internal selection, and path generation. The robustness techniques of [7, 8] provide a starting point for studying the analytic selectors and returns away from the exact integer lattice.

Optical phase transport and fluid circulation suggest possible media in which to investigate winding-based memory. A corresponding device model would have to specify its physical state space and covering, construct the read and write operations, and bound their precision, energy, and execution costs. The discrete-return semantics identifies the mathematical operations that such an implementation would need to realize.

7.3 Open problems

A broader mathematical question is which effective continuation systems admit an analogous combination of retained state, separating finite observations, and recursive access to deeper structure. The present finite calculus gives one complete instance and a general finite-observation obstruction. It provides a precise starting point for studying that question across other analytic coverings.

Within the calculus, further sublanguage classifications can ask which restrictions admit a finite closed stopping observation and which extensions escape it. Quantitative questions concern the size of compiled query trees, the growth of memory codes, and the cost of realizing a return. Reusable formal interfaces for covering transport, analytic observations, and complete-execution refinement could support these investigations across different geometric models.

The present model joins persistent branches, finite analytic control, and access to deeper memory in one finite syntax. Its execution has both an exact partial-recursive characterization and an actual path-lifting realization. The Forward separation makes the contribution of deeper access visible as a theorem about computational power.

AI contributions and author responsibility. Generative AI (OpenAI ChatGPT and Codex, including collaborating AI agents) contributed substantially to the conception and mathematical development of this work, including research ideation, theoretical constructions, conjectures, proofs, Lean formalization, implementation, and manuscript preparation.

The human author directed the research, determined its scope and principles, established its aesthetic aims and the use of Lean as the native research environment, critically evaluated the results, and assumes full responsibility for the work.

The formalization and source code accompany this manuscript for independent verification.

A The fixed input and output maps

This appendix specifies the interface of (5.1) without reference to the function being computed. It is inherited from the fixed alphabet of Mathlib's partial-recursive compiler, and the Monodromy block encoding is applied only after that alphabet is fixed.

Let $K = \{\mathsf{main}, \mathsf{rev}, \mathsf{aux}, \mathsf{stack}\}$ be four track names and $A = \{\mathsf{cons}_\ell, \mathsf{cons}, \mathsf{bit}_0, \mathsf{bit}_1\}$ four stack symbols. Use the finite tape alphabet

$$\Sigma = \{\mathsf{false}, \mathsf{true}\} \times (K \to \operatorname{Option} A).$$

The Boolean marks a stack bottom, and each track stores either a symbol or an empty entry. The blank is the false marker with every entry empty. Fix an enumeration $e : \Sigma \simeq [a]$ sending this blank to zero. This same enumeration is used for every program, input, and output. In Lean it is obtained from a fixed finite enumeration by swapping the blank's index with zero.

For $s \in A$, let $\mathrm{col}(s)$ be the letter with bottom marker false, symbol s on the main track, and empty other tracks. Let $\mathrm{mark}(s)$ be the same letter with marker true. Write $\mathrm{bin}(n)$ for the ordinary binary digits of n in most-significant-first order, with $\mathrm{bin}(0) = []$. The input word is

$$W_n = \mathrm{mark}(\mathsf{cons}) :: \mathrm{map}(\mathrm{col}, \mathrm{bin}(n)), \tag{A.1}$$

interpreting each binary digit as its corresponding stack symbol. This is exactly the compiler's packed word for the singleton numeric list $[n]$: the stack's little-endian bits and final separator are reversed when packed into the tape. Its first separator column carries the bottom marker. The native input is

$$I(n) = \big(e(\mathrm{head}(W_n)),\ 0,\ E_a(\mathrm{map}(e, \mathrm{tail}(W_n)))\big). \tag{A.2}$$

The current symbol occupies the first coordinate because the initial control label is zero in (4.5).

Define a total block decoder $\mathrm{dec}_a : \mathbb{N} \to [a]^*$. Starting from $(v, [])$, take exactly v iterations of this rule: look up the residue modulo 3^a in the finite list $(\delta_j)_{j \in [a]}$; if it names j, replace the state by $(v \div 3^a, j :: w)$; otherwise replace it by $(0, w)$. Finally reverse the accumulated word. Zero has no matching letter, so it remains stationary. The inequalities $0 < \delta_j < 3^a$ give

$$\mathrm{dec}_a(E_a(w)) = w, \qquad |w| \le E_a(w).$$

The second inequality proves that the chosen iteration budget always suffices for valid encodings. The bounded definition gives a total primitive-recursive map on every natural input.

For $x \in \mathbb{Z}^3$, form $w = \mathrm{map}(e^{-1}, \mathrm{dec}_a(\max(x_2, 0)))$. Fold this word from left to right, starting with zero. A main-track bit_b replaces the accumulator z by $2z + b$; every other symbol leaves it unchanged. Call the resulting number $O(x)$. The marker and the other three tracks are ignored. Thus O is total, independent of the source table, and reads the canonical final right half-tape as its binary value. Since appended blank symbols leave the fold unchanged, the readout is well-defined on blank-extended half-tapes. The compiler leaves the separator in the current cell and the numeric bits in the right half-tape at termination, so reading only coordinate two recovers the entire numeric answer.

All constants and finite tables here are fixed. In particular, the input and decoder introduce no source-dependent preprocessing or readout. The formal certificates of primitive recursiveness use a fixed primitive coding of the finite alphabet and of integer vectors.

B Proof correspondence and reproducibility

The project pins Lean to `leanprover/lean4:v4.35.0` and Mathlib to commit

`56a005855ea17e0b0958f73ec801ae140b0a1679`.

Definitions and proofs are in `src/Monodromy/`. The following table lists the principal bridges; names are relative to the `Monodromy` namespace.

Mathematical statement	Formal entry points
Covering transport	`Transport.transport_trans`, `Transport.transport_homotopic`
Logarithmic branches	`Logarithm.transport_turn`
Regular-root selectors	`Choice.kernel_fiber`, `Choice.analyticAt_kernel`, `Choice.kernel_comp`
Lemma 3.1	`CoverExpressions.evalReal_int`, `CoverExpressions.analyticAt_evalReal`, `CoverExpressions.Guard.evalReal_int`, `CoverExpressions.Guard.eval_gate`
Complete native equality	`CoverExpressions.Program.evalReal_init`
Proposition 3.3	`CoverExpressions.Program.run_transport`, `CoverExpressions.query_transport`, `CoverExpressions.query_integer`
Finite ternary tables	`CoverExpressions.Ternary.eval_selectPower`, `CoverExpressions.Ternary.eval_quotientPower`
Block word encoding	`BinaryBlocks.encode_cons`, `BinaryBlocks.encode_injective`, `BinaryBlocks.decode_encode`
Lemma 4.1	`CoverCompiler.stepInt_code`, `CoverCompiler.code_injective`
Theorem 4.2	`TwoStacks.decode_surjective`, `TwoStacks.eval_decode`, `FiniteRenaming.eval_configuration`, `CoverCompiler.evalInt_code`, `CoverCompiler.advance_iterate_realize`, `CoverCompiler.run_transport`; see also `Universality.every_finite_machine` for the direct radix realization
Classical compiler bridge	`FormalCompilation.every_partrec`, `FormalCompilation.eval1`, `FormalCompilation.eval0`, `NativeLowerBound.every_partrec`
Lemma 5.1	`NativeIO.primrec_input`, `NativeIO.primrec_output`, `NativeIO.output_represents`
Lemma 5.3	`IntegerPrimrec.add`, `CoverExpressions.Program.primrec_stepInt`, `CoverExpressions.Program.partrec_evalInt`
Theorem 5.6	`Structure.continuation_characterization`, `Structure.analytic_continuation_realization`, `Structure.computes_iff`
Lemma 5.7	`CoverExpressions.Program.forward_supported`, `CoverExpressions.Program.forward_finite_observation`
Theorem 5.8	`CoverExpressions.Program.forward_halting_bound`, `ForwardBoundary.primrec_halting`, `Structure.descent_strictly_extends`
Corollary 5.9	`Structure.closed_observation_barrier`

Mathematical statement	Formal entry points
Proposition 5.10	`Resolution.iterate_tail_degree_iff`, `Resolution.all_depths_iff`

The formal syntax allows any positive query degree; the predicate `UsesRadix3` selects exactly the cubic expressions of this paper. Its constructors use parameter 2 to specify degree $2 + 1 = 3$. The formal definition `Structure.Computes` uses the result graph of native `evalReal` directly; `computes_iff` proves its equivalence with the complete partial equality (5.2).

The verification commands and axiom audit are given in Section 6. The main equivalence is an existence theorem for a finite realizing program for each partial recursive function; its proof uses finite enumerations and finite-support restrictions. This is the formal compilation interface used in the mathematical results above.

References

[1] Lenore Blum, Mike Shub, and Steve Smale. On a theory of computation and complexity over the real numbers: NP-completeness, recursive functions and universal machines. *Bulletin of the American Mathematical Society (New Series)*, 21(1):1–46, 1989. doi: 10.1090/S0273-0979-1989-15750-9. URL `https://profshub.github.io/articles/1989-On_a_theory_of_computation_and_complexity_over_the_real_numbers.pdf`.

[2] Mario Carneiro. Formalizing computability theory via partial recursive functions. In John Harrison, John O'Leary, and Andrew Tolmach, editors, *10th International Conference on Interactive Theorem Proving (ITP 2019)*, volume 141 of *Leibniz International Proceedings in Informatics*, pages 12:1–12:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. doi: 10.4230/LIPIcs.ITP.2019.12. URL `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2019.12`.

[3] Mario Carneiro and The mathlib Community. Modelling partial recursive functions using Turing machines. mathlib4 computability source, 2020. URL `https://github.com/leanprover-community/mathlib4/blob/56a005855ea17e0b0958f73ec801ae140b0a1679/Mathlib/Computability/TuringMachine/ToPartrec.lean`. Pinned source and imported machine translations; revision specified in Appendix B.

[4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37. URL `https://lean-lang.org/papers/lean4.pdf`.

[5] Michael S. Floater and Tom Lyche. Two chain rules for divided differences and Faà di Bruno's formula. *Mathematics of Computation*, 76(258):867–877, 2007. doi: 10.1090/S0025-5718-06-01916-8.

[6] Otto Forster. *Lectures on Riemann Surfaces*, volume 81 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, 1981. doi: 10.1007/978-1-4612-5961-9. URL `https://link.springer.com/book/10.1007/978-1-4612-5961-9`. Translated by Bruce Gilligan.

[7] Daniel S. Graça and Ning Zhong. Analytic one-dimensional maps and two-dimensional ordinary differential equations can robustly simulate Turing machines. *Computability*, 12(2):117–144, 2023. doi: 10.3233/COM-210381. URL https://arxiv.org/abs/2109.15073.

[8] Daniel S. Graça, Manuel L. Campagnolo, and Jorge Buescu. Robust simulations of Turing machines with analytic maps and flows. In S. Barry Cooper, Benedikt Löwe, and Leen Torenvliet, editors, *New Computational Paradigms*, volume 3526 of *Lecture Notes in Computer Science*, pages 169–179. Springer, 2005. doi: 10.1007/11494645_21. URL https://sqigmath.tecnico.ulisboa.pt/pub/GracaDS/05-GCB-stable.pdf.

[9] Allen Hatcher. *Algebraic Topology*. Cambridge University Press, 2002. ISBN 978-0-521-79540-1. URL https://pi.math.cornell.edu/~hatcher/AT/ATpage.html.

[10] Pascal Koiran and Cristopher Moore. Closed-form analytic maps in one and two dimensions can simulate universal Turing machines. *Theoretical Computer Science*, 210 (1):217–223, 1999. doi: 10.1016/S0304-3975(98)00117-0. URL https://sites.santafe.edu/~moore/pubs/elem.html.

[11] Claude E. Shannon. Mathematical theory of the differential analyzer. *Journal of Mathematics and Physics*, 20(1–4):337–354, 1941. doi: 10.1002/sapm1941201337. URL https://onlinelibrary.wiley.com/doi/10.1002/sapm1941201337.

[12] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. Association for Computing Machinery, 2020. doi: 10.1145/3372885.3373824. URL https://leanprover-community.github.io/papers/mathlib-paper.pdf.